

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 定义顺序表结构
#define MAX_SIZE 100
typedef int ElemType;
typedef struct {
    ElemType data[MAX_SIZE];
    int length;
} SqList;

// 顺序表就地逆置
void reverseSqList(SqList *L) {
    int left = 0;
    int right = L->length - 1;
    while (left < right) {
        ElemType temp = L->data[left];
        L->data[left] = L->data[right];
        L->data[right] = temp;
        left++;
        right--;
    }
}

// 定义单链表结构
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;

// 合并两个递增有序单链表为递减有序单链表
LinkList mergeAndReverse(LinkList A, LinkList B) {
    LinkList C = NULL;
    LinkList tail = NULL;
    LinkList current;
    while (A != NULL || B != NULL) {
        if (A == NULL) {
            current = B;
            B = B->next;
        } else if (B == NULL) {
            current = A;
            A = A->next;
        } else if (A->data < B->data) {
            current = A;
            A = A->next;
        } else {
            current = B;
            B = B->next;
        }
    }
}

```

```

        current->next = C;
        C = current;
    }
    return C;
}

// 求两个递增有序顺序表的交集
void intersectionOfSqlists(SqlList A, SqlList B, SqlList *C) {
    int i = 0, j = 0, k = 0;
    while (i < A.length && j < B.length) {
        if (A.data[i] == B.data[j]) {
            C->data[k++] = A.data[i];
            i++;
            j++;
        } else if (A.data[i] < B.data[j]) {
            i++;
        } else {
            j++;
        }
    }
    C->length = k;
}

```

```

// 删除A表中既在B表又在C表中出现的元素
void deleteSharedItems(SqlList *A, SqlList B, SqlList C) {
    int i, j, k;
    for (i = 0; i < A->length; i++) {
        int foundInB = 0, foundInC = 0;
        for (j = 0; j < B.length; j++) {
            if (A->data[i] == B.data[j]) {
                foundInB = 1;
                break;
            }
        }
        for (k = 0; k < C.length; k++) {
            if (A->data[i] == C.data[k]) {
                foundInC = 1;
                break;
            }
        }
        if (foundInB && foundInC) {
            for (j = i; j < A->length - 1; j++) {
                A->data[j] = A->data[j + 1];
            }
            A->length--;
        } else {
            i++;
        }
    }
}

```

// 删除单向循环链表中某个结点的前驱结点

```
void deletePredecessor(LNode *S) {  
    if (S == NULL || S->next == S) {  
        return;  
    }  
    LNode *prev = S;  
    LNode *current = S->next;  
    while (current->next != S) {  
        prev = current;  
        current = current->next;  
    }  
    prev->next = current->next;  
    free(current);  
}
```

// 判断输入的字符序列是否为回文

```
int isPalindrome() {  
    char stack[MAX_SIZE];  
    int top = -1;  
    char ch;  
    while ((ch = getchar()) != '@') {  
        stack[++top] = ch;  
    }  
    while ((ch = getchar()) != '@') {  
        if (top < 0 || ch != stack[top--]) {  
            return 0;  
        }  
    }  
    return top == -1;  
}
```

// 递归求F(n)

```
int recursiveF(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * recursiveF(n / 2);  
}
```

// 消除递归求F(n)

```
int iterativeF(int n) {  
    int result = 1;  
    while (n > 0) {  
        result *= n;  
        n /= 2;  
    }  
    return result;  
}
```

// 火车车厢调度

```
void trainScheduling(char *cars, int n) {
```

```

char stack[MAX_SIZE];
int top = -1;
int outputIndex = 0;
char output[MAX_SIZE * 2];
for (int i = 0; i < n; i++) {
    if (cars[i] == 'S') {
        output[outputIndex++] = 'S';
    } else {
        stack[++top] = cars[i];
    }
}
while (top >= 0) {
    output[outputIndex++] = stack[top--];
}
output[outputIndex] = '\0';
printf("调度操作序列: %s\n", output);
}

int main() {
    // 顺序表就地逆置测试
    SqList list1 = {.data = {1, 2, 3, 4, 5}, .length = 5};
    reverseSqList(&list1);
    printf("逆置后的顺序表: ");
    for (int i = 0; i < list1.length; i++) {
        printf("%d ", list1.data[i]);
    }
    printf("\n");

    // 合并单链表测试
    LinkList A = (LinkList)malloc(sizeof(LNode));
    A->data = 1;
    A->next = (LinkList)malloc(sizeof(LNode));
    A->next->data = 3;
    A->next->next = NULL;
    LinkList B = (LinkList)malloc(sizeof(LNode));
    B->data = 2;
    B->next = (LinkList)malloc(sizeof(LNode));
    B->next->data = 4;
    B->next->next = NULL;
    LinkList C = mergeAndReverse(A, B);
    printf("合并并逆序后的单链表: ");
    while (C != NULL) {
        printf("%d ", C->data);
        LinkList temp = C;
        C = C->next;
        free(temp);
    }
    printf("\n");

    // 求顺序表交集测试
    SqList listA = {.data = {1, 3, 5, 7}, .length = 4};

```

```

SqList listB = {.data = {3, 5, 7, 9}, .length = 4};
SqList listC;
intersectionOfSqLists(listA, listB, &listC);
printf("顺序表交集: ");
for (int i = 0; i < listC.length; i++) {
    printf("%d ", listC.data[i]);
}
printf("\n");

// 删除A表中共享元素测试
SqList listAA = {.data = {1, 2, 3, 2, 4}, .length = 5};
SqList listBB = {.data = {2, 3}, .length = 2};
SqList listCC = {.data = {2, 4}, .length = 2};
deleteSharedItems(&listAA, listBB, listCC);
printf("删除共享元素后的A表: ");
for (int i = 0; i < listAA.length; i++) {
    printf("%d ", listAA.data[i]);
}
printf("\n");

// 删除单向循环链表前驱结点测试
LNode *nodeS = (LNode *)malloc(sizeof(LNode));
nodeS->data = 3;
LNode *node1 = (LNode *)malloc(sizeof(LNode));
node1->data = 1;
LNode *node2 = (LNode *)malloc(sizeof(LNode));
node2->data = 2;
node1->next = node2;
node2->next = nodeS;
nodeS->next = node1;
deletePredecessor(nodeS);
printf("删除前驱结点后的单向循环链表: ");
LNode *current = nodeS->next;
do {
    printf("%d ", current->data);
    current = current->next;
} while (current != nodeS);
printf("\n");

// 回文判断测试
printf("请输入字符序列(以@结束): ");
if (isPalindrome()) {
    printf("是回文\n");
} else {
    printf("不是回文\n");
}

// 递归与迭代求F(n)测试
int n = 4;
printf("递归求F(%d): %d\n", n, recursiveF(n));
printf("迭代求F(%d): %d\n", n, iterativeF(n));

```

```
// 火车车厢调度测试
char cars[] = {'H', 'S', 'H', 'S', 'H'};
trainScheduling(cars, 5);

return 0;
}
```